

Développement d'une application de gestion administratifs

Cahier des Charges Technique

Partie 1 : Gestion des frais de scolarité

1. Contexte et Objectifs

1.1 Contexte

L'établissement souhaite moderniser et digitaliser la gestion des frais de scolarité pour améliorer le suivi des paiements, réduire les erreurs manuelles et simplifier la relation entre l'administration, les parents et les étudiants.

1.2 Objectifs

- Suivre les frais de scolarité par étudiant
- Générer automatiquement les factures
- Permettre le paiement en ligne ou la saisie des paiements manuels
- Gérer les échéanciers et relances
- Produire des rapports de gestion financière

2. Fonctionnalités attendues

2.1 Gestion des utilisateurs

- Comptes pour : administration, comptabilité, parents/tuteurs, étudiants
- Rôles et permissions (lecture seule, édition, validation...)
- Interface parent/étudiant pour consulter les factures et effectuer les paiements

2.2 Suivi des frais

- Création de grilles tarifaires selon : classe, niveau, bourse, nationalité...

- Association des frais à un étudiant ou un groupe
- Gestion des réductions (bourses, remises)

2.3 Facturation

- Génération automatique ou manuelle des factures (mensuelles, annuelles...)
- Possibilité d'ajouter des frais ponctuels (ex : frais d'inscription)
- Téléchargement des factures au format PDF
- Numérotation automatique des factures

2.4 Paiements

- Enregistrement des paiements manuels (espèces, chèque, virement)
- Intégration avec une passerelle de paiement en ligne (Stripe, PayPal, etc.)
- Historique des paiements par étudiant/famille
- Remboursements

2.5 Relances et échéanciers

- Création d'échéanciers de paiement personnalisés
- Envoi automatique de rappels par e-mail/SMS pour les factures impayées

2.6 Rapports et statistiques

- Rapport global des encaissements (par période, classe, étudiant...)
- Liste des impayés
- Statistiques par année scolaire
- Export Excel/PDF des données financières

3. Contraintes techniques

3.1 Plateformes

- Application web responsive (bureau, tablette, smartphone)
- Portail en ligne pour les familles

3.2 Technologies proposées (exemple)

- Backend : Firebase
- Frontend : React / Vue.js
- Base de données : Firestore
- Intégration API de paiement (Stripe, PayPal, etc.)

3.3 Sécurité

- Accès sécurisé par identifiant/mot de passe
- Connexion HTTPS
- Protection des données personnelles (RGPD)
- Sauvegardes automatiques des données

4. Livrables attendus

- Application prête à l'emploi
- Manuel utilisateur (administration + famille)
- Documentation technique pour maintenance

Partie 2 : Gestion des documents administratifs

1. Présentation du projet

1.1. Objectif général

Développer une application web/mobile permettant de gérer efficacement les documents administratifs pour l'école Maroc YNOV Campus : attestation d'inscriptions, attestation de réussite, bulletin, etc.

1.2. Objectifs spécifiques

- Numérisation et archivage sécurisé des documents
- Accès aux documents pour les utilisateurs autorisés (administration, parents, étudiant)
- Génération automatisée de documents
- Suivi des demandes administratives

2. Environnement technique

2.1. Plateforme cible

- Web (navigateur desktop et mobile) ----- pour l'administration
- Éventuellement application mobile (Android / iOS) ----- pour parents et étudiant

2.2. Technologies suggérées

- **Backend** : Firebase
- **Frontend** : React / Vue.js
- **Base de données** : Firestore
- **Hébergement** : Serveur dédié ou cloud (AWS, Azure, OVH, etc.)
- **Sécurité** : HTTPS, Authentification JWT / OAuth2, rôles utilisateurs

3. Fonctionnalités attendues

3.1. Gestion des utilisateurs

- Création et gestion des profils (admin, personnel, parents, étudiant)
- Authentification sécurisée
- Attribution de rôles et permissions

3.2. Gestion des documents

- Téléversement de documents (PDF, Word, etc.)
- Catégorisation : certificats, bulletins, conventions, attestations ...
- Génération automatique de documents à partir de modèles
- Archivage, classement par année scolaire, étudiant, type de document
- Recherche avancée

3.3. Gestion des demandes

- Formulaire de demande de documents
- Suivi du traitement des demandes (statuts : en cours, validé, rejeté)
- Notifications (email / application) pour les étapes importantes

3.4. Tableau de bord administratif

- Statistiques : nombre de documents, demandes traitées...
- Alertes sur les documents manquants ou en attente

3.5. Journalisation et traçabilité

- Historique des actions par utilisateur
- Système de logs consultable par les administrateurs

5. Livrables attendus

- Maquettes de l'interface utilisateur (UI/UX)
- Code source de l'application
- Base de données (structure + script d'installation)
- Documentation technique (installation, configuration, API)
- Guide utilisateur

Partie 3 : Gestion des absences

1. Contexte et Objectifs

1.1 Contexte

L'établissement Maroc YNOV Campus souhaite mettre en place une application permettant de gérer efficacement les absences de ses employés, étudiants et professeurs.

1.2 Objectifs

- Automatiser la gestion des absences (maladie, congés, retards, etc.)
- Permettre aux utilisateurs de déclarer une absence en ligne
- Faciliter la validation des absences par les responsables
- Générer des rapports et statistiques
- Garantir la traçabilité et l'archivage des données

2. Fonctionnalités attendues

2.1 Gestion des utilisateurs

- Création de comptes (employé, RH, manager, admin)
- Authentification sécurisée (login/mot de passe, 2FA optionnel)
- Gestion des rôles et permissions

2.2 Saisie et suivi des absences

- Formulaire de demande d'absence (motif, dates, justificatif, etc.)
- Ajout de pièces jointes (certificat médical, etc.)
- Historique des absences par utilisateur
- Système de validation (workflow RH/manager)

2.3 Notifications

- Alertes par e-mail pour nouvelle demande, validation ou refus
- Rappels pour les absences prolongées/non justifiées

2.4 Administration

- Tableau de bord RH/admin
- Recherche multicritère (par nom, période, motif...)
- Statistiques : taux d'absence, répartition, etc.
- Export des données (PDF, Excel)

3. Contraintes techniques

3.1 Plateformes

- Web responsive (PC, tablette, mobile)
- Possibilité d'une version mobile (iOS, Android) en option

3.2 Technologies souhaitées (à adapter selon vos besoins)

- Backend : Firebase
- Frontend : React / Vue.js
- Base de données : Firestore
- Hébergement : cloud ou serveur interne

3.3 Sécurité

- RGPD conforme
- Gestion des accès par rôle
- Journalisation des actions (logs)

4. Livrables attendus

- Application fonctionnelle
- Documentation utilisateur et technique
- Cahier de tests et validation
- Formation des utilisateurs (si nécessaire)